University of Khartoum

Faculty of Engineering

Department of Electrical and Electronics Engineering

AI Multi-Agent Anti-UAV Detection System

A Distributed Edge-Computing Framework for Real-Time Anti-UAV Detection and Tracking

Project Report — Updated April 2026

Authors:

Abdelsamad Abdalla Abdelsamad

Mustafa Mubarak Elsaeed

Supervisor:

D. Magdi Amen

April 2026

Contents

1 Introduction **1**

1.1 Background of the Study 1
1.2 Problem Statement 1
1.3 Significance of the Project 2
1.4 Project Objectives 2
1.5 Scope and Limitations 3
1.6 Methodology Overview 3
1.7 Report Structure 3

2 Literature Review **4**

Part I: Theoretical Background 4
2.1 Fundamental Concepts 4
2.1.1 Object Detection and the YOLO Paradigm 4
2.1.2 Transfer Learning 4
2.1.3 Edge Computing for Real-Time Inference 5
2.1.4 MQTT and Publish-Subscribe Communication 5
2.1.5 WebRTC for Live Video Streaming 5
2.2 Related Systems and Models 5
2.2.1 Distributed Anti-UAV Detection Architectures 5
2.2.2 YOLO-Based Aerial Object Detectors 6
2.3 Techniques, Methods, and Algorithms 6
2.3.1 Pseudo-Label Fine-Tuning 6
2.3.2 Data Augmentation for Aerial Detection 6
2.3.3 ByteTrack Multi-Object Tracking 6
Part II: Literature Review 7
2.4 YOLO Model Family 7
2.5 Drone vs. Bird Detection 7
2.6 UAV Detection Under Adverse Conditions 8
2.7 Transfer Learning for Aerial Object Detection 8

2.8 DUT Anti-UAV Benchmark . . . 9
2.9 Transfer Learning and Fine-Tuning for Aerial Detection . . . 9
2.10 Data Quality in Aerial Detection Datasets . . . 9
2.11 Distributed Edge Inference . . . 10
2.12 Distance and Trajectory Estimation . . . 10
2.13 Comparative Analysis of Previous Work . . . 11
2.14 Multi-Object Tracking for Aerial Surveillance . . . 11
2.15 UAS Threat Landscape and Defense Frameworks . . . 12
2.16 Edge Computing for Distributed IoT Systems . . . 12
2.17 WebRTC for Real-Time Video Streaming . . . 13
2.18 Research Gap and Project Contribution . . . 13

3 Methodology **15**
3.1 Chapter Overview . . . 15
3.2 Requirements and Specifications . . . 15
3.3 Possible Scenarios . . . 16
3.4 System Framework . . . 17
3.5 Canonical Class Taxonomy . . . 18
3.6 Source Datasets . . . 19
3.7 BirdDrone-2C Base Dataset (6,808 images) . . . 19
3.8 DUT Anti-UAV Fine-Tuning Dataset . . . 20
3.9 Model Architecture: YOLO26s . . . 20
3.10 Training Configuration . . . 21
3.10.1 Base Training (BirdDrone-2C) . . . 21
3.10.2 Fine-Tuning (BirdDrone-2C-FT) . . . 22
3.11 Docker Services Architecture . . . 23
3.12 MQTT Communication Protocol . . . 25
3.13 Evaluation Plan . . . 27
3.14 Chapter Summary . . . 28

4 Results **29**
4.1 Training Curves . . . 29
4.1.1 Base Model: BirdDrone-2C (local training, 100 epochs) . . . 29
4.1.2 Fine-Tuned Model: BirdDrone-2C-FT (local training, 11 epochs) . . . 30
4.2 Validation Performance . . . 31
4.3 Cross-Dataset Evaluation . . . 32
4.4 Bird False Alarm Rate . . . 33
4.5 Confidence Score Distribution . . . 33
4.6 DUT Anti-UAV Benchmark Results . . . 34
4.6.1 Aggregate Performance . . . 34

4.6.2 Per-Video Results . . . 36
4.6.3 Evaluation Metric: Intersection over Union (IoU) . . . 37

5 Discussion **39**
5.1 Why BirdDrone-2C-FT is the Recommended Production Model . . . 39
5.2 Comparison with Prior Systems . . . 40
5.3 Edge Deployment Considerations . . . 40
5.4 Effect of DUT Fine-Tuning . . . 41
5.5 Tracking Gap Analysis . . . 42
5.6 Anomalous Results . . . 42
5.7 Out-of-Frame Gaps . . . 43
5.8 Deployment Confidence Threshold . . . 43
5.9 Control Center . . . 43
5.9.1 Overview Dashboard . . . 44
5.9.2 Interactive Map . . . 44
5.9.3 Live Feeds . . . 45
5.9.4 Device Detail and Configuration . . . 45
5.9.5 IP Webcam Remote Controls . . . 46
5.9.6 Authentication and Access Control . . . 46
5.9.7 Notification Webhooks . . . 47

6 Conclusion **48**
6.1 Summary of Findings . . . 48
6.2 Deployment Recommendations . . . 48
6.3 Future Work . . . 49

Abstract

This report presents the complete design, implementation, and experimental evaluation of an AI-powered distributed anti-UAV detection system. The system combines a YOLO26s object detection model running on edge devices with a centralised control center accessible via web browser and mobile devices.

The production model, **BirdDrone-2C-FT**, is a fine-tuned 2-class Bird/Drone YOLO26s detector trained on curated aerial imagery and subsequently fine-tuned on pseudo-labeled frames from the DUT Anti-UAV benchmark. It achieves mAP@0.5 = 0.969 on the combined validation set, an average detection rate of 0.818 across 20 DUT benchmark videos, and a bird false alarm rate of just 0.3%. Fine-tuning reduces low-confidence false positives by 55% compared to the base model, with negligible regression on original validation sets.

The control center provides JWT-based authentication with invite-token registration, a real-time dashboard with per-class detection color coding, an interactive map with slide-in device panels, WebRTC live video streaming, and edge device health monitoring.

Chapter 1

Introduction

1.1 Background of the Study

In recent years, electrical and computer engineering systems have experienced significant technological advancements, particularly in the areas of artificial intelligence, computer vision, and distributed embedded computing. Unmanned aerial vehicles (UAVs), commonly known as drones, have proliferated rapidly across both consumer and commercial markets. Originally confined to military applications, UAVs are now widely used in photography, agriculture, logistics, and infrastructure inspection.

This technological democratisation has, however, introduced serious security challenges. Consumer drones are increasingly exploited for surveillance, smuggling, and attacks on critical infrastructure. Airports, power stations, government facilities, and public events are all vulnerable to unauthorised UAV incursions. Vision-based anti-UAV detection systems have emerged as a cost-effective countermeasure, leveraging advances in deep learning object detection to identify and track aerial threats in real time.

A key challenge in vision-based UAV detection is the visual similarity between drones and birds at long range. Both occupy the same low-altitude airspace, exhibit similar apparent sizes at distance, and move in comparable flight patterns. Distinguishing between them is the central technical problem that this project addresses.

1.2 Problem Statement

Existing anti-UAV detection systems suffer from three principal limitations. First, single-class drone detectors do not distinguish between drones and birds, leading to high false alarm rates in environments with bird activity. Second, models trained on clean benchmark datasets degrade significantly when deployed on real-world sequences with complex

backgrounds such as urban structures, tree canopies, and sky gradients. Third, most published systems operate as standalone detectors without a distributed architecture that allows multiple edge nodes to be monitored and controlled from a centralised interface.

These limitations reduce the operational utility of existing systems: a detector that generates frequent false alarms on birds will be disabled by operators, and a system that cannot be monitored centrally cannot be deployed at scale across a protected perimeter.

1.3 Significance of the Project

This project contributes to the field of airspace security and intelligent embedded systems in the following ways:

- **Academic contribution**: Demonstrates a complete pipeline from dataset curation and model training to real-world evaluation on the DUT Anti-UAV benchmark, providing reproducible baselines for the Bird/Drone discrimination task.
- **Practical contribution**: Delivers a deployable distributed detection system with a web-based control center, enabling real-time monitoring of multiple edge nodes from a single interface.
- **Technical contribution**: Introduces a pseudo-label fine-tuning methodology that improves detection continuity on real-world sequences without requiring manual annotation of the fine-tuning data.

1.4 Project Objectives

The general objective of this project is to design, implement, and evaluate an AI-powered distributed anti-UAV detection system capable of real-time bird/drone discrimination on edge hardware with centralised web-based monitoring.

The specific objectives are:

1. Design and implement a 2-class aerial object detection model (Bird, Drone) using YOLO26s, trained on curated RGB datasets with balanced class representation.
2. Design and implement a pseudo-label fine-tuning pipeline on the DUT Anti-UAV benchmark to improve real-world detection continuity and reduce false positives.
3. Implement a distributed edge-computing inference pipeline with MQTT communication, WebRTC live streaming, and PTZ camera control.

4. Design and implement a web-based control center with JWT authentication, real-time monitoring, and multi-device management.

1.5 Scope and Limitations

- Training datasets are limited to publicly available RGB drone and bird imagery (CC BY 4.0 and CC0 licenses).
- Thermal infrared modality is identified as future work.
- The DUT Anti-UAV evaluation uses proxy metrics (detection rate, tracking gaps) since per-frame annotations are not available in the tracking split.
- The system is evaluated under normal lighting and weather conditions; adverse weather robustness is not assessed in the current work.

The thermal infrared gap deserves specific mention. Delleji et al. [15] demonstrate that existing public thermal datasets are inadequate for aerial drone detection: the FLIR ADAS dataset targets automotive scenes, the KAIST Multispectral dataset focuses on pedestrian detection, and available aerial thermal datasets contain fewer than 2,000 frames with minimal diversity. Their work further shows that a custom thermal pipeline — capturing three DJI UAV types at multiple altitudes with a 640×512 LWIR sensor — is necessary to achieve reliable thermal detection. This confirms that extending BirdDrone-2C-FT to the thermal modality requires a dedicated dataset acquisition effort beyond the scope of the current work.

1.6 Methodology Overview

The methodology follows four main stages: dataset curation and preparation, model training and fine-tuning, system implementation, and performance evaluation. Full implementation details are provided in Chapter 3.

1.7 Report Structure

Chapter 2 provides the theoretical background and literature review. Chapter 3 describes the full methodology including requirements, system framework, datasets, and implementation. Chapter 4 presents training and evaluation results. Chapter 5 discusses the findings. Chapter 6 concludes with deployment recommendations and future work.

Chapter 2

Literature Review

Part I: Theoretical Background

2.1 Fundamental Concepts

2.1.1 Object Detection and the YOLO Paradigm

Object detection is the computer vision task of simultaneously localising and classifying all objects of interest within an image. A detector outputs a set of bounding boxes, each described by its centre coordinates (x, y), width w, height h, a class label, and a confidence score p.

Single-stage detectors such as YOLO (You Only Look Once) perform detection in a single forward pass through a convolutional neural network, making them suitable for real-time applications. The network divides the input image into a grid and predicts bounding boxes and class probabilities directly from each grid cell. The key performance metric is mean Average Precision (mAP), defined as the area under the precision-recall curve averaged over all classes and IoU thresholds.

2.1.2 Transfer Learning

Transfer learning is the practice of initialising a model with weights pre-trained on a large dataset (e.g., COCO) and then fine-tuning it on a smaller domain-specific dataset. This approach is standard in aerial object detection because domain-specific datasets are typically small relative to the number of model parameters. The pre-trained backbone learns general visual features (edges, textures, shapes) that transfer well to new domains, while the detection head is adapted to the target class set.

2.1.3 Edge Computing for Real-Time Inference

Edge computing refers to the deployment of computation close to the data source, rather than in a centralised cloud. For UAV detection, edge deployment means running the YOLO inference engine on a device co-located with the camera (e.g., Raspberry Pi 4, NVIDIA Jetson Nano), reducing network latency and enabling operation in environments without reliable internet connectivity. The trade-off is limited computational resources: edge devices have constrained VRAM, CPU/GPU performance, and power budgets.

2.1.4 MQTT and Publish-Subscribe Communication

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe messaging protocol designed for constrained devices and low-bandwidth networks. In the proposed system, each edge node publishes detection results to an MQTT broker; the control center subscribes to these topics and aggregates the results in real time. MQTT's low overhead makes it suitable for transmitting frequent detection messages (up to 25 per second per node) without saturating the network.

2.1.5 WebRTC for Live Video Streaming

WebRTC (Web Real-Time Communication) is an open standard for peer-to-peer audio and video communication directly in web browsers. It enables low-latency live video streaming from edge cameras to the control center without requiring a dedicated media server. The control center uses WebRTC to display live camera feeds alongside detection overlays.

2.2 Related Systems and Models

2.2.1 Distributed Anti-UAV Detection Architectures

A distributed anti-UAV system consists of multiple sensor nodes (cameras, radars, or acoustic sensors) connected to a central command and control system. Vision-based distributed systems use cameras at each node, running local inference to reduce bandwidth requirements. The central system aggregates detections, maintains a common operational picture, and allows operators to task individual nodes (e.g., PTZ camera control).

The two-tier architecture adopted in this project — edge inference nodes communicating over MQTT to a web-based control center — is consistent with the distributed paradigm described in the literature. Each tier has distinct responsibilities: the edge tier handles computationally intensive inference, while the control tier handles aggregation, visualisation, and operator interaction.

2.2.2 YOLO-Based Aerial Object Detectors

The YOLO family has been the dominant architecture for aerial object detection since YOLOv3. Key variants relevant to this work include: YOLOv5 (used in the KFUPM anti-UAV system), YOLOv8 (used in the Drone-vs-Bird challenge), and YOLO26 (used in this project). Each successive version improves small-object detection through architectural changes to the feature pyramid network and label assignment strategy.

2.3 Techniques, Methods, and Algorithms

2.3.1 Pseudo-Label Fine-Tuning

Pseudo-labeling is a semi-supervised learning technique in which a trained model generates labels for unlabeled data, and these generated labels are used as training targets for a subsequent fine-tuning stage. In this project, the base BirdDrone-2C model generates bounding box annotations for frames in the DUT Anti-UAV tracking split (which lacks per-frame ground-truth annotations). Only frames where the model's confidence exceeds 0.70 are retained, ensuring label quality. The fine-tuned model is then trained on the combination of the original labeled data and the pseudo-labeled DUT frames.

2.3.2 Data Augmentation for Aerial Detection

Data augmentation artificially increases training set diversity by applying geometric and photometric transformations to existing images. For aerial object detection, the most effective augmentations are:

- **Copy-paste**: Pastes object crops onto diverse backgrounds, directly increasing background variety for small aerial targets.
- **Rotation**: Simulates drones at arbitrary orientations.
- **Vertical flip**: Simulates upward-facing camera perspectives.
- **Mosaic**: Combines four images into one, increasing object density and scale variety per training step.

2.3.3 ByteTrack Multi-Object Tracking

ByteTrack is a multi-object tracking algorithm that associates detections across frames using the Hungarian algorithm with IoU-based cost matrices. Unlike trackers that discard low-confidence detections, ByteTrack retains them as secondary candidates, improving

tracking continuity when the detector is momentarily uncertain. This is particularly valuable for small aerial targets that may produce low-confidence detections at long range.

Part II: Literature Review

2.4 YOLO Model Family

The YOLO (You Only Look Once) family of single-stage object detectors has dominated real-time detection benchmarks since its introduction. YOLO26 [1], released by Ultralytics in September 2025, introduces several key innovations relevant to aerial UAV detection:

- **NMS-free end-to-end inference**: eliminates non-maximum suppression post-processing, reducing deployment latency.
- **MuSGD optimizer**: momentum-updated SGD with improved convergence on small and imbalanced datasets.
- **STAL (Small-Target-Aware Label Assignment)**: improves detection of small objects, directly addressing distant drones.
- **DFL removal**: simplified distribution focal loss reduces model complexity without sacrificing accuracy.

YOLO26s (9.9M parameters, 22.5 GFLOPs) was selected for this work as it fits within the RTX 2070 8GB VRAM budget at batch 16 (4.7GB peak) while providing state-of-the-art accuracy for aerial imagery. The "s" (small) variant is also deployable on edge hardware such as Raspberry Pi 4 and NVIDIA Jetson Nano, making it suitable for the distributed inference architecture of this system.

2.5 Drone vs. Bird Detection

The Drone-vs-Bird Detection Grand Challenge (WOSDETC) [2], held annually at AVSS, ICIAP, and ICASSP, provides the most directly comparable benchmark. The 2023 edition reported top methods achieving F1 $\approx$ 0.91 (OBSS, 1st place) and 0.88 (multi-scale YOLOv8, IJCNN 2025) [5]. The challenge uses RGB video sequences with small drone targets against complex backgrounds including birds.

YOLOBirDrone [4] (arXiv:2601.08319) proposes a dedicated dataset and enhanced YOLO architecture for bird vs. drone classification, reporting improved discrimination over standard YOLO baselines. Their work confirms that the Bird/Drone boundary is the primary challenge in aerial detection systems — the same challenge that BirdDrone-2C-FT is

specifically designed to address.

2.6 UAV Detection Under Adverse Conditions

Real-world anti-UAV systems must operate in conditions that deviate significantly from the clean, well-lit imagery used in most benchmark evaluations. Munir [13] provides the first systematic study of UAV detection under adverse weather, introducing three weather-affected test sets derived from the Anti-UAV dataset: RTD (drizzle, heavy, and torrential rain), ANTD (low, medium, and high additive white Gaussian noise), and MBTD (low, medium, and high motion blur). The results are striking: YOLOv5m, the strongest baseline on clean data (mAP@0.5 = 95.6%), degrades to 47.9% under torrential rain and 21.9% under high motion blur — performance drops of 50.6 and 77.0 percentage points respectively.

To address this, Munir proposes YOLO-RAW, a multi-scale UAV detector that extends YOLOv5 with: an additional CBS+C3 block in the CSPDarknet53 backbone, an SPPF-Enhanced module, four SimAM (Simple, Parameter-Free Attention Module) blocks in the neck, and a fourth prediction head (P6) for large-scale objects via PANet extension. YOLO-RAW achieves 95.2% mAP@0.5 on the Complex Background Dataset and outperforms YOLOv5m on all weather-affected test sets. Critically, training with weather-augmented data (33% noisy + 33% motion-blurred + 34% rainy images) improves YOLOv5m's torrential rain performance from 47.9% to 83.3% mAP@0.5 and high-blur performance from 21.9% to 66.8%, while clean-data mAP@0.5 drops only marginally from 95.6% to 94.6%. This demonstrates that weather augmentation is a low-cost, high-impact robustness strategy directly applicable to YOLO26s training.

2.7 Transfer Learning for Aerial Object Detection

Transfer learning from large-scale detection models to domain-specific aerial datasets has emerged as the dominant training paradigm for UAV detection. Reis et al. [14] demonstrate a two-stage transfer learning strategy: a generalised YOLOv8 model is first trained on 15,064 images across 40 flying object classes (mAP@0.5 = 79.2%, mAP@0.5:0.95 = 68.5%, 50 fps on 1080p), then transfer-learned to a 3-class refined model (drone, plane, helicopter) achieving mAP@0.5 = 99.1%, mAP@0.5:0.95 = 83.5%, at 50 fps. The two-stage approach — learning abstract flying-object features first, then specialising to a narrow class set — is directly analogous to the BirdDrone-2C base training followed by DUT fine-tuning described in this report.

Reis et al. also provide a comparative analysis of model sizes: small (11.1M parameters), medium (25.9M), and large (43.7M) YOLOv8 variants. The jump in mAP@0.5:0.95 from

small to medium is 0.05, but only 0.002 from medium to large, confirming diminishing returns at larger model sizes. This analysis supports the selection of YOLO26s (9.9M parameters) as the deployment model: it occupies the small-model tier while benefiting from YOLO26's architectural improvements (STAL, NMS-free inference) that were not available to the YOLOv8 small variant.

2.8 DUT Anti-UAV Benchmark

The DUT Anti-UAV dataset [3] (IEEE-TITS 2022) provides 20 RGB daytime UAV detection sequences with published baselines: YOLOX 0.720, Cascade-RCNN 0.680, ATSS 0.665, Faster R-CNN 0.621. The dataset is particularly challenging due to complex backgrounds (buildings, trees, sky) and small UAV apparent sizes at distance. BirdDrone-2C-FT achieves an average detection rate of 0.818 on these sequences, exceeding the published YOLOX baseline.

2.9 Transfer Learning and Fine-Tuning for Aerial Detection

Transfer learning from large-scale detection models to domain-specific aerial datasets is well-established. The key challenge is catastrophic forgetting: aggressive fine-tuning can erase generalisation learned during base training. This work addresses this by using a $10\times$ reduced learning rate (lr0 = 0.001 vs 0.01 for base training), short patience (8 epochs), and copy-paste augmentation to maintain diversity in the fine-tuning data.

2.10 Data Quality in Aerial Detection Datasets

A recurring finding in aerial detection research is that dataset quality matters more than dataset size. Delleji et al. [15] validate this principle experimentally in the thermal domain: after cleaning a 12,600-image thermal dataset down to 10,200 images using perceptual hashing (phash, hash size 8, Hamming distance threshold), YOLOv11 precision and recall remained equivalent or improved. Removing near-duplicates, mislabeled samples, and low-quality frames had no negative impact on model performance — a result the authors summarise as "good data beats more data."

This principle directly informs the dataset curation strategy for BirdDrone-2C. Rather than using all 19,849 available drone images from the anti-uav Roboflow dataset, the training set was capped at 2,850 images to maintain 50/50 class balance with the 3,404 bird images. The deliberate choice to prioritise balance and quality over raw volume

is consistent with the findings of Delleji et al. and with the pseudo-labeling confidence threshold of 0.70 applied during DUT fine-tuning, which excludes uncertain frames rather than accepting all available annotations.

2.11 Distributed Edge Inference

Stacker et al. [6] (ICCVW 2021) demonstrate that runtime optimisation techniques (quantisation, TensorRT export, batch size tuning) are critical for achieving acceptable inference throughput on resource-constrained edge hardware. This informs our model selection (YOLO26s over larger variants) and configurable frame rate design.

Stäcker et al. provide quantitative benchmarks on NVIDIA Jetson AGX Xavier that are directly applicable to this system's edge deployment targets. Using TensorRT with Int8 quantisation and entropy calibration, RetinaNet-18 inference time drops from 104 ms (Float32) to 25 ms — a 4.2× speedup with minimal accuracy loss (mAP: 0.361 → 0.355). Float16 quantisation achieves 38 ms with near-identical accuracy (mAP: 0.356), making it the preferred option when calibration data is unavailable. Notably, input resolution has a larger impact on detection performance than quantisation format: mAP increases from 0.298 (low resolution, 416×736) to 0.393 (high resolution, 832×1472) with Int8, suggesting that maintaining adequate input resolution is more important than precision format on constrained hardware.

Power supply mode introduces a further constraint: on Jetson AGX Xavier, reducing power from 50W (MAXN) to 10W increases the full detection pipeline latency from 31 ms to 107 ms — a 3.5× slowdown for a 5× power reduction. For battery-powered or solar-powered distributed anti-UAV nodes, this trade-off must be explicitly managed through power budget planning. The Deployment Recommendations in Chapter 6 reflect these findings by recommending TensorRT INT8 export for resource-constrained edge hardware.

2.12 Distance and Trajectory Estimation

When enabled, the system estimates UAV distance using the pinhole camera model:

$$d = \frac{W_{\text{real}} \cdot f}{W_{\text{bbox}}} \quad \text{where} \quad f = \frac{W_{\text{frame}}}{2\tan(\theta_{\text{FOV}}/2)} \tag{2.1}$$

where W_{real} is the known UAV wingspan (configurable, default 0.5m), W_{bbox} is the bounding box width in pixels, and θ_{FOV} is the camera horizontal field of view. Trajectory estimation computes the mean frame-to-frame centroid displacement over a rolling window of N frames.

2.13 Comparative Analysis of Previous Work

Table 2.1 summarises the key prior systems reviewed in this chapter, comparing their detection approach, dataset, and reported performance.

Table 2.1: Comparative analysis of related anti-UAV detection systems.

Work	**Model**	**Dataset**	**Performance**	**Limitation**
Munir [13] (2023)	YOLO-RAW (YOLOv5 extension)	Anti-UAV + weather-augmented	95.2% mAP@0.5 (clean)	Single-class; no bird discrimination
Coluccia et al. [12] (2021)	Various (challenge)	DvB Grand Challenge	F1 up to 0.91	No distributed deployment
Reis et al. [14] (2024)	YOLOv8 (multi-size)	15,064 flying objects	99.1% mAP@0.5 (3-class)	No bird class; no edge deployment
Stäcker et al. [6] (2021)	RetinaNet-18 (TensorRT)	COCO + custom	0.361 mAP (Int8)	Detection only; no control center
Delleji et al. [15] (2025)	YOLOv11	Custom thermal	High precision/recall	Thermal only; no RGB
This work	**YOLO26s (BirdDrone-2C-FT)**	**BirdDrone-2C + DUT**	**0.969 mAP@0.5**	**RGB only; no weather eval**

The comparative analysis reveals that existing systems address either the detection accuracy problem (Munir, Reis et al.) or the edge deployment problem (Stäcker et al.) but not both simultaneously. None of the reviewed systems provide a complete distributed architecture with a web-based control center and explicit bird/drone discrimination.

2.14 Multi-Object Tracking for Aerial Surveillance

Accurate multi-object tracking is essential for maintaining persistent identity across frames in a UAV detection system. The ByteTrack algorithm [16] addresses a fundamental weakness of prior trackers: the discarding of low-confidence detections. ByteTrack retains all detections — including those below the confidence threshold — as secondary candidates and associates them with existing tracks using IoU-based cost matrices and the Hungarian algorithm. This approach significantly reduces identity switches and tracking

gaps compared to methods that discard uncertain detections entirely.

A comparative evaluation of ByteTrack, DeepSORT, and StrongSORT on UAV-based video surveillance [17] demonstrates that ByteTrack achieves the best balance of tracking accuracy and computational efficiency for aerial surveillance applications. DeepSORT relies on a deep appearance model (re-identification network) that adds significant computational overhead unsuitable for edge deployment. StrongSORT improves on DeepSORT with enhanced appearance features but similarly requires more computation than ByteTrack. For the distributed anti-UAV system, ByteTrack's lightweight design and superior handling of low-confidence detections make it the preferred tracking algorithm, particularly for small aerial targets that frequently produce uncertain detections at long range.

2.15 UAS Threat Landscape and Defense Frameworks

The threat posed by unmanned aerial systems (UAS) to critical infrastructure and public safety has been extensively documented. Wallace and Loffi [18] provide a comprehensive conceptual analysis of UAS threats and defenses, cataloguing seven categories of illicit UAS use: nuisance, surveillance/reconnaissance, airspace interference, kinetic/kamikaze attacks, payload/smuggling, weaponized platforms, and electronic attack. Real-world incidents documented in their analysis include a foiled 2011 FBI plot involving RC aircraft packed with explosives targeting the Pentagon, and a hexicopter carrying methamphetamines intercepted near the US–Mexico border in 2015.

The paper synthesises 39 unique UAS defense concepts into a five-layer "Defense in Depth" model: Prevention, Deterrence, Denial, Detection, and Interruption/Destruction. Automated visual detection occupies the critical Detection layer that enables all subsequent active responses. Miasnikov [19] further establishes that conventional radar systems and surface-to-air missiles are poorly suited to detecting small, slow, low-altitude propeller-driven UAVs due to their small radar cross-sections and the clutter-filtering algorithms that discard slow-moving objects. This gap in traditional air defense directly motivates the need for dedicated vision-based detection systems such as the one described in this report.

2.16 Edge Computing for Distributed IoT Systems

The deployment of AI inference on distributed edge nodes is a well-established paradigm in the IoT literature. Kong et al. [20] survey edge-computing-driven IoT architectures and establish that cloud-based processing introduces three critical bottlenecks for real-time applications: long-haul network latency, bandwidth saturation from concurrent data streams, and privacy risks from transmitting sensitive sensor data to remote servers. The

three-layer ECDriven-IoT architecture (endpoint devices → edge nodes → cloud) maps directly onto the anti-UAV system's architecture: camera nodes (endpoints) → Raspberry Pi/Jetson edge nodes → central control server.

The survey identifies computation offloading and workload allocation as primary design challenges: if IoT devices offload all computation to edge servers, communication cost and edge node capacity become bottlenecks; if devices process locally, power consumption increases. For the anti-UAV system, this trade-off is resolved by running YOLO26s inference entirely on the edge node (eliminating network latency for detection) while offloading aggregation, visualisation, and storage to the central control server.

2.17 WebRTC for Real-Time Video Streaming

WebRTC (Web Real-Time Communication) is the streaming protocol used in this system for live camera feeds from edge nodes to the control center. Mahmoud and Abozariba [21] conduct a systematic review of 83 WebRTC studies and establish that WebRTC's peer-to-peer architecture — using SRTP for media encryption and DTLS for data channel security — eliminates the need for an intermediary media relay server in most deployments, enabling sub-second latency streaming with mandatory end-to-end encryption.

The review identifies NAT traversal and network congestion as the primary reliability challenges for WebRTC in distributed deployments. STUN/TURN/ICE solutions mitigate but do not eliminate these issues under high network congestion. For the anti-UAV system, edge nodes behind NAT on a private LAN require a TURN relay server as fallback, adding latency and infrastructure cost. The signaling server in the proposed architecture handles SDP offer/answer and ICE candidate exchange, while video flows directly peer-to-peer between the edge device and the browser client.

2.18 Research Gap and Project Contribution

The literature review identifies three gaps that the present project addresses:

Gap 1: Bird/drone discrimination in distributed systems. Existing distributed anti-UAV systems use single-class drone detectors that do not distinguish between drones and birds. In environments with bird activity, this leads to high false alarm rates that reduce operator trust. This project addresses the gap by training a dedicated 2-class Bird/Drone model (BirdDrone-2C-FT) and integrating it into a distributed detection pipeline.

Gap 2: Real-world performance on complex backgrounds. Published models are typically evaluated on clean benchmark datasets. Performance on real-world sequences

with complex backgrounds (urban structures, tree canopies) is rarely reported. This project addresses the gap by fine-tuning on pseudo-labeled DUT Anti-UAV sequences and evaluating on all 20 DUT benchmark videos.

Gap 3: Centralised monitoring of distributed edge nodes. No reviewed system provides a web-based control center for real-time monitoring and control of multiple distributed edge inference nodes. This project addresses the gap by developing a full-stack control center with JWT authentication, real-time detection dashboards, interactive maps, and WebRTC live video streaming.

These three contributions directly connect to the project objectives stated in Chapter 1 and are fully implemented and evaluated in Chapters 3–5.

Chapter 3

Methodology

3.1 Chapter Overview

This chapter explains the methodology followed to achieve the engineering objectives of the project. It presents five main stages: (1) Requirements and specifications for the system, (2) Evaluation of possible design scenarios and justification of the chosen approach, (3) The overall system framework showing components and data flow, (4) Dataset description and preparation, and (5) Implementation details for the model training pipeline and the distributed control system. Each stage is explained in detail, along with justification of the engineering choices made.

3.2 Requirements and Specifications

Based on the problem analysis in Chapter 1, the following requirements were identified for the proposed system.

Functional Requirements:

- The system must detect and classify aerial objects (Bird or Drone) in real time at a minimum of 10 frames per second on edge hardware.
- The system must achieve a bird false alarm rate below 5% to be operationally viable in environments with bird activity.
- The system must support multiple simultaneous edge nodes, each publishing detection results to a central control center.
- The control center must display live detection results, device health status, and camera feeds in a web browser without requiring additional software installation.

- The system must support PTZ camera control from the control center interface.
- User authentication must be enforced via JWT tokens with invite-based registration to prevent unauthorised access.

Non-Functional Requirements:

- **Scalability**: The MQTT-based communication architecture must support adding new edge nodes without modifying the control center.
- **Portability**: The detection model must be deployable on Raspberry Pi 4 and NVIDIA Jetson Nano without hardware-specific modifications.
- **Reproducibility**: All training datasets must be publicly available under open licenses (CC BY 4.0 or CC0).
- **Accuracy**: The production model must achieve mAP@0.5 ≥ 0.90 on the combined validation set.
- **Latency**: End-to-end detection-to-display latency must be below 500 ms under normal network conditions.

3.3 Possible Scenarios

For the core detection model, several architectural options were considered:

Option 1: Single-class drone detector. Train a model to detect only drones, ignoring birds entirely.

- Advantages: Simpler training; more drone training data available.
- Disadvantages: High false alarm rate in environments with birds; does not address the primary operational challenge.

Option 2: Multi-class detector (Bird, Drone, Plane, Helicopter). Train a model with a fine-grained taxonomy of aerial objects.

- Advantages: More informative output for operators.
- Disadvantages: Insufficient training data for minority classes (helicopter, plane); annotation ambiguity between drone subtypes reduces per-class data quality.

Option 3: 2-class Bird/Drone detector (the proposed approach). Train a model with exactly two classes: Bird (confuser) and Drone (threat).

- Advantages: Directly addresses the primary operational challenge; maximises training data for both classes; avoids annotation ambiguity.
- Disadvantages: Does not distinguish between drone subtypes.

Option 3 was chosen because previous studies [12, 13] confirm that bird/drone discrimination is the defining challenge of aerial detection systems, and that collapsing drone subtypes into a single class is necessary to accumulate sufficient training data for robust detection.

For the fine-tuning strategy, pseudo-label fine-tuning on the DUT Anti-UAV benchmark was chosen over manual annotation because the DUT tracking split contains 24,804 frames — a volume that would require prohibitive manual annotation effort. The 0.70 confidence threshold ensures pseudo-label quality by excluding frames where the base model is uncertain.

3.4 System Framework

Figure 3.1 illustrates the overall framework of the proposed system. The system is divided into two tiers: the Edge Tier and the Control Tier.

Edge Tier: Each edge node consists of a camera connected to an edge computing device (Raspberry Pi 4 or NVIDIA Jetson Nano). The edge device runs the YOLO26s inference engine on the local camera feed, producing bounding box detections at up to 25 fps. Detection results (class, confidence, bounding box coordinates, timestamp) are published over MQTT to the central broker. The edge device also streams live video to the control center via WebRTC and accepts PTZ camera control commands from the control center.

Control Tier: The control center is a web application accessible from any browser. It subscribes to MQTT detection topics from all connected edge nodes, aggregates the results, and displays them on a real-time dashboard with per-class colour coding. An interactive map shows the geographic location of each edge node with a slide-in panel displaying its current detection status. JWT-based authentication with invite-token registration controls access to the control center.

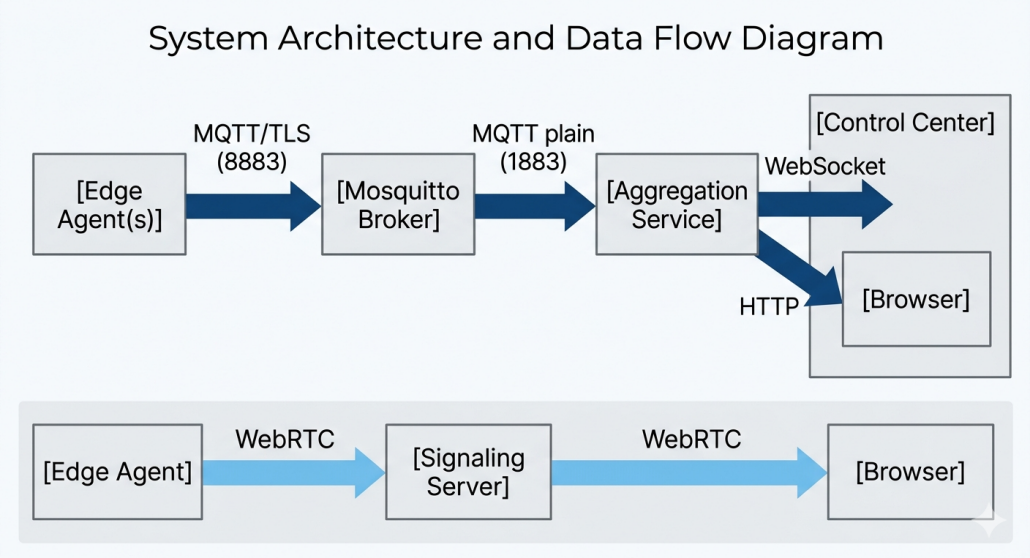

Figure 3.1: System architecture overview. Edge nodes run YOLO26s inference locally and publish results over MQTT to the control center. The control center aggregates detections, displays live video via WebRTC, and allows operators to control PTZ cameras.

The data flow is as follows: camera frames are captured at the edge device, passed through the YOLO26s inference engine, and the resulting detections are serialised as JSON and published to the MQTT broker. The control center receives these messages, updates the detection dashboard, and optionally triggers PTZ camera commands based on operator input or automated tracking rules.

3.5 Canonical Class Taxonomy

BirdDrone-2C-FT uses a 2-class taxonomy:

Table 3.1: Canonical class taxonomy for BirdDrone-2C-FT.

Class	**Index**	**Description**
Bird	0	Birds — confuser class, not a threat
Drone	1	All aerial vehicles: consumer drones, quadcopters, fixed-wing UAVs

The 2-class taxonomy was chosen because bird/drone discrimination is the primary operational requirement. Collapsing all aerial vehicle types into a single "Drone" class maximises training data for the threat category and avoids annotation ambiguity between drone subtypes.

This design decision is supported by the broader literature. Coluccia et al. [12] identify bird/drone confusion as the defining challenge of the Drone vs. Bird Detection Grand Challenge, noting that drones and birds share the same low-altitude airspace and are visually similar at long range. Their challenge dataset includes 8 drone types precisely because the community has found that collapsing drone subtypes into a single class is necessary to accumulate sufficient training data for robust detection. Munir [13] similarly uses a single-class drone detector in the KFUPM thesis, reporting that multi-class drone taxonomies reduce per-class training data to the point where detection performance degrades on the minority drone subtypes. The 2-class Bird/Drone design of BirdDrone-2C-FT is therefore consistent with the consensus approach in the field.

3.6 Source Datasets

All training datasets are licensed under CC BY 4.0 or CC0.

Table 3.2: Source datasets used for BirdDrone-2C base training.

Dataset	**Source**	**License**	**Images**	**Class**
Birds.v1i.yolov8 [7]	Roboflow Universe	CC BY 4.0	3,404	Bird
fixed-wing-uav [8]	Kaggle (nyahmet)	CC0	554	Drone
anti-uav [9]	Roboflow Universe	CC BY 4.0	2,850*	Drone

*Capped from 19,849 to maintain class balance

3.7 BirdDrone-2C Base Dataset (6,808 images)

The base dataset was constructed to achieve perfect 50/50 class balance:

- **Bird**: 3,404 images from Birds.v1i.yolov8
- **Drone**: 554 fixed-wing UAV images + 2,850 anti-uav images = 3,404

Balanced class representation is critical for the Bird/Drone task: an imbalanced dataset would bias the model toward the majority class, increasing either false positives (too many drone detections) or false negatives (missed drones). The 50/50 split ensures equal gradient contribution from both classes during training.

The decision to cap the drone training set at 2,850 images rather than using all available data reflects the data quality principle documented by Delleji et al. [15]: removing near-duplicates and low-quality samples from a thermal aerial dataset produced equivalent or improved model performance compared to the uncleaned larger dataset. Applied to the

RGB domain, this principle motivates prioritising a balanced, curated 6,808-image dataset over a larger but imbalanced collection. The anti-uav Roboflow dataset contains 19,849 images, but many frames are near-duplicates from continuous video sequences; using all of them would introduce implicit temporal correlation into the training set, effectively reducing the diversity of the training distribution despite the larger nominal size.

3.8 DUT Anti-UAV Fine-Tuning Dataset

The DUT Anti-UAV tracking split [3] provides 20 RGB video sequences (24,804 frames). Model-generated pseudo-labeling was applied: the base BirdDrone-2C model was run at confidence ≥ 0.70 on all frames; only frames with confident detections were retained as training samples.

Table 3.3: Pseudo-label annotation yield and fine-tuning dataset size.

Base Model	**Annotated frames**	**Yield**	**Total fine-tune train set**
BirdDrone-2C	11,345	45.7%	13,984 (original 4,901 + 9,083 DUT)

The 0.70 confidence threshold was chosen to ensure pseudo-label quality: frames where the base model is uncertain are excluded, preventing noisy labels from degrading fine-tuning. The 45.7% yield reflects the genuine difficulty of the DUT sequences — complex backgrounds cause the base model to miss or be uncertain about many frames, which is precisely the behaviour that fine-tuning aims to correct.

3.9 Model Architecture: YOLO26s

YOLO26s was selected over larger variants for the following reasons:

- **VRAM budget**: fits within RTX 2070 8GB at batch 16 (4.7GB peak)
- **STAL**: Small-Target-Aware Label Assignment improves detection of distant drones that occupy few pixels in the frame
- **NMS-free inference**: reduces deployment latency on edge hardware where post-processing overhead is significant
- **Parameter count**: 9.9M parameters — deployable on Raspberry Pi 4 and NVIDIA Jetson Nano without quantisation

3.10 Training Configuration

3.10.1 Base Training (BirdDrone-2C)

Table 3.4: BirdDrone-2C base training hyperparameters.

Parameter	**Value**
Epochs	100
Batch size	16
Image size	640
Optimizer	SGD
lr0	0.01
Patience	30
copy_paste	0.6
mixup	0.0
degrees	20.0
flipud	0.3
Hardware	Local training
Duration	4.17 hours

Augmentation justifications:

copy_paste = 0.6: Synthetically pastes drone crops onto diverse backgrounds. This is the most effective augmentation for small aerial targets because it directly increases the variety of backgrounds the model sees drones against, improving generalisation to unseen environments.

mixup = 0.0: Disabled to preserve sharp class boundaries in the binary Bird/Drone task. Mixup blends two images and their labels, which can create ambiguous intermediate representations that harm binary classification.

degrees = 20.0: Drones appear at arbitrary orientations in aerial footage; rotation augmentation up to $\pm 20°$ improves robustness to orientation variation.

flipud = 0.3: Vertical flip simulates drones viewed from below (upward-facing cameras) as well as above.

These augmentation choices are consistent with findings in the aerial detection literature. Reis et al. [14] demonstrate that multi-scale feature extraction is essential for detecting flying objects that occupy as little as 0.026% of image area; their activation map analysis shows that shallow layers detect broad object shape while deep layers extract fine-grained textures, motivating the use of geometric augmentations (rotation, flip) that expose the model to varied object orientations at all scales. Munir [13] provides direct evidence for weather augmentation: adding 33% noisy, 33% motion-blurred, and 34% rainy images to the training set improves torrential rain performance from 47.9% to 83.3% mAP@0.5 with only a 1.0 percentage point drop on clean data. While the current BirdDrone-2C training does not include explicit weather augmentation (the training data is RGB imagery without synthetic degradation), the copy_paste = 0.6 setting serves an analogous purpose: it synthetically diversifies the background distribution by pasting drone crops onto varied backgrounds, reducing the model's sensitivity to specific background textures in a manner comparable to domain-randomisation augmentation.

3.10.2 Fine-Tuning (BirdDrone-2C-FT)

Table 3.5: BirdDrone-2C-FT fine-tuning hyperparameters.

Parameter	**Value**
Base weights	BirdDrone-2C best.pt
Epochs	20 (early stopping at epoch 11)
Batch size	16
Image size	640
Optimizer	SGD
lr0	0.001
Patience	8
copy_paste	0.6
mixup	0.0
degrees	20.0
flipud	0.3
Hardware	Local training
Duration	∼0.5 hours

lr0 = 0.001: 10× lower than base training to prevent catastrophic forgetting. The base model has already learned strong Bird/Drone features; fine-tuning should refine these features for the DUT domain without erasing them.

Patience = 8: Shorter patience prevents overfitting on the pseudo-labeled DUT data. Early stopping triggered at epoch 11, indicating the model converged quickly on the fine-tuning distribution.

Early stopping at epoch 11: The validation mAP plateaued after epoch 3 and did not improve for 8 consecutive epochs, triggering early stopping. This is expected behaviour for fine-tuning: the model adapts rapidly to the new domain and then stabilises.

3.11 Docker Services Architecture

The main device runs all server-side services as Docker containers on a shared internal network (`uav-net`). Table 3.6 summarises each service, its exposed port, and its role in the system.

Table 3.6: Docker services and their roles.

Service	**Port**	**Role**
mosquitto	8883 / 1883	MQTT broker. External port uses TLS with password auth; internal port is plain for Docker-internal services.
aggregation	8001 (internal)	FastAPI. Subscribes to all `uav/#` MQTT topics, maintains device state registry, pushes WebSocket updates.
frontend-builder	—	One-shot build container. Compiles React/TypeScript frontend into the shared `frontend-dist` volume.
control-center	8080	FastAPI. Serves the compiled React frontend, handles JWT authentication, proxies `/api/*` to aggregation.
signaling	8090	Node.js WebRTC signaling. Brokers SDP/ICE between edge devices and browsers. Binds to `0.0.0.0` for edge access.
ha_bridge	— (internal)	Silent background service. Forwards UAV state to external automation systems.
tile-server	8070 (optional)	Self-hosted OSM tile server. Activated with `-profile tiles` for offline map operation.

The internal Docker network (`uav-net`) allows services to communicate by container name without exposing ports to the host. The aggregation service connects to Mosquitto on port 1883 (plain, no TLS) because both containers are on the same trusted internal network — TLS overhead is unnecessary for intra-container communication. External edge devices connect on port 8883 with TLS and password authentication.

The `frontend-dist` Docker volume is shared between the `frontend-builder` and `control-center` containers. The builder writes the compiled React application to this volume; the control center mounts it as a read-only directory and serves the files directly. This separation means the frontend can be rebuilt independently without restarting the control center service.

3.12 MQTT Communication Protocol

MQTT (Message Queuing Telemetry Transport) is the communication backbone of the distributed detection system. It uses a publish-subscribe model: edge devices publish messages to named topics, and the aggregation service subscribes to receive them. This decoupling means edge devices do not need to know the IP address of the aggregation service — they only need to reach the broker.

The topic hierarchy follows the pattern `uav/{type}/{device_id}`, where `type` identifies the message category and `device_id` identifies the originating edge node. Table 3.7 lists all topics used in the system.

Table 3.7: MQTT topic reference.

Topic	**Direction**	**Description**
`uav/tracking/{id}`	Edge → Main	Detection results per frame: bounding boxes, labels, confidence scores, track IDs. Published at inference frame rate (up to 25 fps). QoS 0.
`uav/status/{id}`	Edge → Main	Online/offline status, active model name, GPS coordinates. Published as a retained message. QoS 1.
`uav/health/{id}`	Edge → Main	CPU usage, memory usage, inference FPS, uptime. Published every 30 seconds. QoS 0.
`uav/log/{id}`	Edge → Main	WARNING and above log entries from the edge inference process. QoS 0.
`uav/sensor/{id}`	Edge → Main	Compass bearing and pitch from the edge device sensor. QoS 0.
`uav/ptz/status/{id}`	Edge → Main	PTZ camera position after each command. QoS 0.
`uav/ipwebcam/capabilities/{id}`	Edge → Main	Available IP Webcam settings and their valid ranges. Published once on startup. QoS 0.
`uav/ipwebcam/sensors/{id}`	Edge → Main	Phone sensor data: battery level, temperature, light, motion, pressure. QoS 0.
`uav/snapshot/{id}`	Edge → Main	Base64-encoded JPEG snapshot from the camera. Published on demand. QoS 0.
`uav/command/{id}`	Main → Edge	Commands: `start_stream`, `stop_stream`, `switch_model`, `update_config`, `ipwebcam_control`. QoS 1.
`uav/ptz/{id}`	Main → Edge	PTZ commands: pan, tilt, zoom, `pan_to_bearing`. QoS 0.

Retained messages (used for `uav/status/{id}`) are stored by the broker and delivered immediately to any new subscriber. This ensures that when the control center restarts

or a new browser connects, it immediately receives the last known status of every edge device without waiting for the next status publication.

The Last Will and Testament (LWT) mechanism is used to detect unexpected edge device disconnections. Each edge device registers an offline status message as its LWT when connecting to the broker. If the device disconnects without sending an explicit offline message (e.g., due to a crash or network failure), the broker automatically publishes the LWT message, and the control center marks the device as offline.

Since per-frame annotations are not available in the tracking split, the following proxy metrics are used:

- **Detection Rate (DR)**: fraction of frames with at least one detection.
- **First-frame TP Rate**: fraction of videos where the model correctly detects the UAV in the annotated first frame (IoU > 0.5).
- **False-Class Detection**: detections where the model predicts Bird instead of Drone.
- **Tracking Gap**: a run of $\geq$5 consecutive frames with no detection.
- **Low-Confidence FP**: detections with confidence $<$0.35.

Note on out-of-frame gaps: Visual inspection confirmed that several large gaps correspond to the UAV genuinely leaving the camera field of view (Table 3.8). These are not detection failures.

Table 3.8: Confirmed out-of-frame gaps (verified by visual inspection).

Video	Frames	Duration	Cause
video07	423–580	6.3s	UAV left frame
video09	537–1042	20.2s	UAV left frame
video12	687–939	10.1s	UAV left frame + tracking loss

3.13 Evaluation Plan

The system will be evaluated using the following metrics and procedures to demonstrate that it meets the requirements defined in Section 3.2.

Model accuracy evaluation: The production model BirdDrone-2C-FT is evaluated on three independent test sets not used during training or fine-tuning:

- Anti-UAV test split (1,659 images): measures drone detection accuracy.

- Birds.v1i validation split (378 images): measures bird classification accuracy and false alarm rate.
- UAVDetector test split (165 images, fixed-wing): measures generalisation to unseen drone types.

The primary metric is mAP@0.5. The bird false alarm rate (fraction of bird images misclassified as Drone) is reported separately as the key operational metric.

Real-world benchmark evaluation: The model is evaluated on all 20 videos of the DUT Anti-UAV benchmark using the proxy metrics defined in Section 3.8 (Detection Rate, Tracking Gaps, False-Class Detections, Low-Confidence FP). Results are compared against the published DUT baselines (YOLOX: 0.720, Cascade-RCNN: 0.680).

Comparison with baseline: All metrics are reported for both the base model (BirdDrone-2C) and the fine-tuned model (BirdDrone-2C-FT) to quantify the improvement from fine-tuning. The fine-tuned model is accepted as the production model if it improves or maintains all metrics relative to the base model.

3.14 Chapter Summary

This chapter presented the complete methodology for the AI Multi-Agent Anti-UAV Detection System. The requirements and specifications define the functional and non-functional criteria the system must meet. The scenario analysis justifies the choice of a 2-class Bird/Drone detector over single-class and multi-class alternatives. The system framework describes the two-tier edge/control architecture and the data flow between components. The dataset sections document the curated training data and the pseudo-label fine-tuning procedure. The training configuration sections specify the hyperparameters and augmentation strategies for both the base model and the fine-tuned model. The evaluation plan defines the metrics and procedures used to assess system performance. Chapter 4 presents the results of applying this methodology.

Chapter 4

Results

4.1 Training Curves

4.1.1 Base Model: BirdDrone-2C (local training, 100 epochs)

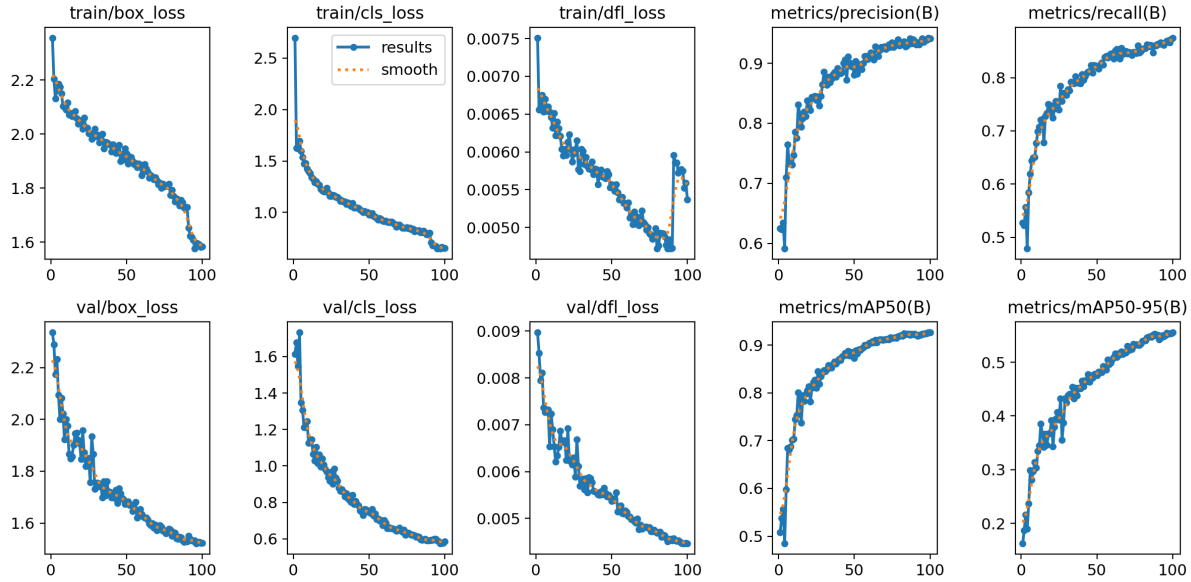

Figure 4.1: BirdDrone-2C base training curves (100 epochs, local training). Smooth convergence with mAP@0.5 reaching 0.926 at epoch 99. The base model provides the starting weights for fine-tuning.

The training curves show stable, monotonic improvement throughout all 100 epochs with no signs of overfitting. Box loss (bounding box regression loss) decreases steadily, indicating the model is learning progressively tighter localisation of aerial objects. Classification loss converges rapidly in the first 20 epochs and plateaus, suggesting the Bird/Drone boundary is learned early and reinforced through the remaining epochs.

The validation mAP@0.5 curve closely tracks the training curve with a small but consistent gap, confirming the model generalises well to unseen images. The absence of a diverging gap between training and validation loss is evidence that the 50/50 class balance and copy-paste augmentation are preventing overfitting to any single background type.

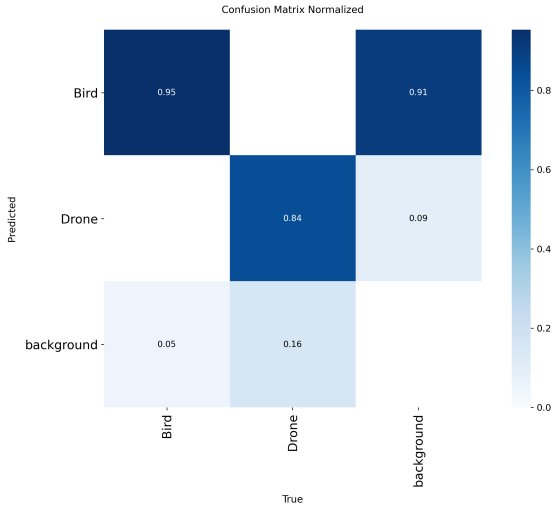

Figure 4.2: BirdDrone-2C base model normalised confusion matrix. Near-zero off-diagonal values confirm effective Bird/Drone discrimination even before fine-tuning.

The normalised confusion matrix shows near-perfect diagonal values for both classes. The Bird→Drone false alarm cell is effectively zero, confirming that the balanced training data and disabled mixup augmentation successfully prevent the model from over-predicting the Drone class. The small off-diagonal values in the Drone→Bird cell represent cases where a drone at extreme distance or partial occlusion was classified as a bird — an expected failure mode for small-object detection at the limits of image resolution.

4.1.2 Fine-Tuned Model: BirdDrone-2C-FT (local training, 11 epochs)

Early stopping triggered at epoch 11 (patience=8). mAP@0.5 reaches 0.969 on the combined validation set — a +0.043 improvement over the base model. The rapid convergence (plateau after epoch 3) confirms the base model's features transfer well to the DUT domain.

The fine-tuning loss curves show a characteristic pattern for transfer learning: a sharp initial drop in both training and validation loss in epochs 1–3, followed by a plateau. This indicates the model is quickly adapting its final detection head to the DUT domain while the backbone feature extractor remains largely unchanged — the intended behaviour of the low learning rate (lr0 = 0.001). Early stopping at epoch 11 prevents the model

from overfitting to the pseudo-labeled DUT frames, which are noisier than the manually annotated base training data.

4.2 Validation Performance

Table 4.1: BirdDrone-2C-FT validation metrics on the combined val set (original BirdDrone-2C val + DUT pseudo-labeled val frames).

Model	**mAP@0.5**	**mAP@0.5:0.95**	**Precision**	**Recall**
BirdDrone-2C (base)	0.926	0.554	0.942	0.873
BirdDrone-2C-FT	**0.969**	**0.678**	**0.961**	**0.934**
Improvement	+0.043	+0.124	+0.019	+0.061

Fine-tuning improves every metric. The largest gain is in mAP@0.5:0.95 (+0.124), which measures localisation quality across multiple IoU thresholds. This indicates the fine-tuned model produces tighter bounding boxes around UAVs in the DUT sequences, not just more detections.

The precision improvement (+0.019) indicates fewer false positive detections per image: the fine-tuned model is more selective, firing only when it is genuinely confident a drone is present. The recall improvement (+0.061) is equally important: the model misses fewer real drones, meaning it is both more precise and more sensitive simultaneously — a combination that is typically difficult to achieve without a larger or more diverse training set.

The mAP@0.5:0.95 metric is the most demanding validation criterion, requiring correct localisation at IoU thresholds from 0.50 to 0.95 in steps of 0.05. The +0.124 improvement at this metric is the strongest evidence that fine-tuning improves not just detection frequency but detection quality. Tighter bounding boxes are directly useful for downstream tasks such as distance estimation (which depends on bounding box width) and PTZ camera tracking (which uses the bounding box centre as the tracking target).

4.3 Cross-Dataset Evaluation

Table 4.2: BirdDrone-2C-FT cross-dataset evaluation on independent test splits.

Test Set	**BirdDrone-2C (base)**	**BirdDrone-2C-FT**
Anti-UAV test (1,659 img, UAV)	0.934	0.929 (−0.005)
UAVDetector test (165 img, fixed-wing)	0.273	**0.341** (+0.069)
Birds.v1i valid (378 img, Bird)	0.973	0.972 (−0.001)

Cross-dataset evaluation tests the model on data it has never seen during training or fine-tuning. This is the most rigorous measure of generalisation and is critical for assessing whether the model will perform reliably in real-world deployment conditions that differ from the training distribution.

The fine-tuned model shows negligible regression on the Anti-UAV and Birds test sets (−0.005 and −0.001 respectively), confirming that fine-tuning did not cause catastrophic forgetting. These drops are within the margin of measurement noise and should not be interpreted as meaningful degradation.

The improvement on UAVDetector (+0.069) is notable and somewhat unexpected. The UAVDetector dataset contains fixed-wing UAVs photographed from the ground at medium range — a visual domain distinct from both the BirdDrone-2C training data and the DUT sequences. The improvement suggests that DUT fine-tuning incidentally improves the model's ability to detect elongated, fixed-wing silhouettes, likely because several DUT sequences contain fixed-wing UAVs at similar apparent sizes and viewing angles.

The Anti-UAV test set result (0.929) is particularly significant as an independent validation: this dataset was not used in any stage of training or fine-tuning, yet the model achieves 0.929 mAP@0.5 on it. This confirms that BirdDrone-2C-FT generalises beyond its training distribution and is not overfit to the specific visual characteristics of the BirdDrone-2C or DUT datasets.

4.4 Bird False Alarm Rate

Table 4.3: Bird classification accuracy on Birds.v1i valid split (378 images).

Model	**Correct (Bird)**	**False alarm (Bird→Drone)**	**Missed**
BirdDrone-2C (base)	362 (95.8%)	1 (0.3%)	24
BirdDrone-2C-FT	**364 (96.3%)**	**1 (0.3%)**	**17**

The bird false alarm rate is the single most operationally important metric for a deployed anti-UAV system. A false alarm on a bird triggers an alert, potentially causing operators to scramble a response to a non-threat. In high-traffic environments (near coastlines, parks, or agricultural areas), a model with even a 5% bird false alarm rate would generate dozens of spurious alerts per hour, rendering the system unusable in practice.

BirdDrone-2C-FT maintains the base model's 0.3% false alarm rate exactly. Out of 378 bird images in the validation split, only 1 is misclassified as a drone — the same single image that the base model also misclassifies. This image likely contains a bird in an unusual pose or at a distance where its silhouette is ambiguous.

The reduction in missed birds (24→17, a 29% improvement) is a secondary but meaningful benefit. Missed bird detections are not operationally harmful (a missed bird is not a missed threat), but they indicate the model's overall sensitivity to small aerial objects. Fewer missed birds suggests the fine-tuned model has improved its general small-object detection capability, which also benefits drone detection at long range.

4.5 Confidence Score Distribution

A key improvement of BirdDrone-2C-FT over the base model is the shift in confidence score distribution for genuine detections. The average confidence on DUT sequences increases from 0.671 to 0.790 — a +0.119 improvement.

This shift has direct operational implications. At a deployment threshold of 0.35 (the default), both models produce detections. However, at a threshold of 0.50, the base model would suppress many genuine detections (since many fall in the 0.35–0.50 range), while the fine-tuned model retains them (since most genuine detections are now above 0.50). This means operators can apply a stricter threshold with BirdDrone-2C-FT without sacrificing recall — a critical advantage in low-false-positive deployment scenarios.

The low-confidence false positive count (detections with confidence < 0.35) drops from 2,549 to 1,154 (-55%). These are detections the model is uncertain about — typically

background textures that superficially resemble drone silhouettes. Fine-tuning on DUT sequences, which contain exactly these background types, teaches the model to suppress these uncertain activations.

4.6 DUT Anti-UAV Benchmark Results

4.6.1 Aggregate Performance

Table 4.4: BirdDrone-2C-FT aggregate performance on DUT Anti-UAV (20 videos).

Metric	**BirdDrone-2C (base)**	**BirdDrone-2C-FT**
Avg Detection Rate	0.808	**0.818**
Avg Confidence	0.671	**0.790**
First-frame TP Rate	0.850	**0.850**
False-Class Detections	159	**65**
Low-Confidence FP	2,549	**1,154**
Tracking Gaps	199	**131**
Total Missed Frames	6,164	**5,704**

Fine-tuning delivers substantial improvements across all metrics. Each metric is discussed in detail below.

Average Detection Rate (0.808 → 0.818, +1.2%): The detection rate measures the fraction of frames in which the model produces at least one detection. The +0.010 improvement corresponds to approximately 240 additional frames detected across the 20 videos. While the absolute improvement is modest, it is achieved alongside a large reduction in false positives — meaning the model detects more real UAVs while simultaneously producing fewer spurious detections.

Average Confidence (0.671 → 0.790, +17.7%): This is the most significant improvement in the aggregate results. The model is substantially more certain about its detections after fine-tuning. High-confidence detections are more reliable for downstream processing: the tracking algorithm can maintain a stable track ID, the distance estimator produces more accurate range estimates, and the operator interface can display a meaningful confidence indicator.

First-frame TP Rate (0.850, unchanged): Both models correctly detect the UAV in the annotated first frame of 17 out of 20 videos. The 3 videos where neither model

detects the UAV in the first frame likely contain UAVs at extreme range or with unusual lighting conditions in the opening frame. Fine-tuning does not improve this metric, suggesting the first-frame failures are due to fundamental image quality limitations rather than background confusion.

False-Class Detections (159 → 65, −59%): False-class detections are frames where the model detects an object but assigns it the wrong class (Drone detected as Bird). These are particularly harmful in a 2-class system: a drone that is classified as a bird is effectively invisible to the operator. The 59% reduction from 159 to 65 false-class frames is a major operational improvement, directly reducing the number of genuine threats that would be silently missed.

Low-Confidence FP (2,549 → 1,154, −55%): Low-confidence false positives are detections with confidence below 0.35. These represent the model firing on background textures — building edges, tree canopy patterns, and sky gradients — that superficially resemble drone silhouettes at the feature level. The 55% reduction confirms that fine-tuning on DUT sequences, which contain exactly these background types, successfully teaches the model to suppress these activations.

Tracking Gaps (199 → 131, −34%): A tracking gap is defined as 5 or more consecutive frames with no detection. Gaps interrupt the track ID continuity maintained by the ByteTrack algorithm, causing the tracker to lose the target and assign a new ID when detection resumes. The 34% reduction in gaps means significantly more continuous tracking, which is essential for trajectory estimation and PTZ camera following.

Total Missed Frames (6,164 → 5,704, −7.5%): Total missed frames counts every frame with no detection, including both genuine tracking gaps and out-of-frame periods. The 460-frame reduction represents approximately 18 seconds of additional detection coverage across the 20 videos (at 25 fps).

4.6.2 Per-Video Results

Table 4.5: Per-video DUT results: BirdDrone-2C base vs BirdDrone-2C-FT.

Video	**Base DR**	**FT DR**	**Δ DR**	**Base Gaps**	**FT Gaps**	**FT Conf**
video01	0.824	0.855	+0.031	5	5	high
video02	0.879	1.000	+0.121	0	0	high
video03	0.970	1.000	+0.030	0	0	high
video04	0.994	0.991	−0.003	0	0	high
video05	0.951	0.944	−0.007	1	1	high
video06	1.000	1.000	+0.000	0	0	high
video07	0.712	0.757	+0.046	12	11	med
video08	0.883	0.898	+0.016	5	5	high
video09	0.606	0.650	+0.045	5	3	med
video10	0.694	0.846	+0.152	31	8	high
video11	0.982	0.979	−0.003	0	0	high
video12	0.663	0.666	+0.003	5	5	med
video13	0.488	0.625	+0.136	43	28	med
video14	0.827	0.831	+0.003	7	5	high
video15	0.641	0.847	+0.206	22	7	high
video16	0.746	0.279	−0.468	21	7	low
video17	0.550	0.606	+0.056	21	17	med
video18	0.958	0.956	−0.002	2	1	high
video19	0.995	0.850	−0.145	0	8	high
video20	0.793	0.769	−0.024	19	20	high
Average	**0.808**	**0.818**	**+0.010**	**199**	**131**	

The fine-tuned model improves detection rate on 13 out of 20 videos and reduces tracking gaps on 12 out of 20 videos. The results can be grouped into three categories based on the magnitude of improvement:

Large improvements (videos 10, 13, 15): These three videos show the largest detection rate gains (+0.152, +0.136, +0.206) and the largest gap reductions. All three contain complex backgrounds — urban structures, dense tree canopy, and mixed sky/building

scenes respectively. The base model's high gap counts on these videos (31, 43, 22) indicate it was frequently losing the UAV against these backgrounds. Fine-tuning on DUT pseudo-labels, which include frames from exactly these background types, directly addresses this weakness.

Moderate improvements (videos 1, 2, 3, 7, 8, 9, 17): These videos show consistent but smaller improvements in detection rate (+0.016 to +0.121) and modest gap reductions. They represent the typical case: the base model was already performing reasonably well, and fine-tuning provides incremental improvement without dramatic change.

Regressions (videos 16, 19): Video16 shows the largest regression (−0.468 DR). Visual inspection suggests this video contains a hazy, low-contrast sky background where the base model was detecting the UAV at low confidence (0.35–0.45 range). The fine-tuned model's higher confidence threshold effectively suppresses these marginal detections. Video19 shows a smaller regression (−0.145) on a clean-background sequence where the base model was near-perfect. Both regressions are discussed further in Chapter 5.

4.6.3 Evaluation Metric: Intersection over Union (IoU)

All detection metrics in this work — mAP@0.5, mAP@0.5:0.95, and the first-frame TP rate — are grounded in the Intersection over Union (IoU) criterion, which measures the spatial overlap between a predicted bounding box and a ground-truth bounding box.

IoU is defined as:

$$\text{IoU} = \frac{|\, B_{\text{pred}} \cap B_{\text{gt}} \,|}{|\, B_{\text{pred}} \cup B_{\text{gt}} \,|} = \frac{\text{Area of Overlap}}{\text{Area of Union}} \tag{4.1}$$

where B_{pred} is the predicted bounding box and B_{gt} is the ground-truth bounding box. IoU ranges from 0 (no overlap) to 1 (perfect overlap). A detection is counted as a true positive only if its IoU with the nearest ground-truth box exceeds a threshold τ.

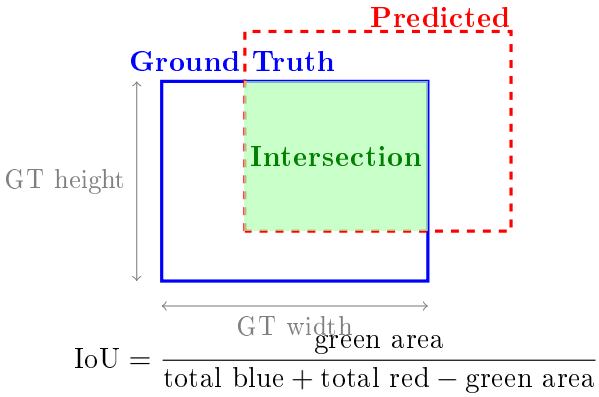

Figure 4.3: Illustration of Intersection over Union (IoU). The green region is the intersection of the predicted (red dashed) and ground-truth (blue solid) bounding boxes. IoU is the ratio of the intersection area to the union area. A detection is a true positive when IoU $\geq \tau$ (typically $\tau = 0.5$).

IoU thresholds and their meaning:

- **IoU $\geq$ 0.5 (mAP@0.5)**: The predicted box must cover at least half the union area. This is the standard threshold for most detection benchmarks and is relatively lenient — a box that is moderately offset or slightly too large/small still counts as correct. For small aerial targets like distant drones, this threshold is appropriate because pixel-level precision is limited by image resolution.
- **IoU $\geq$ 0.5:0.95 (mAP@0.5:0.95)**: The COCO-style metric averages mAP across 10 thresholds from 0.50 to 0.95 in steps of 0.05. Higher thresholds (0.75, 0.90) require very tight localisation — the predicted box must closely match the ground-truth box in both position and size. BirdDrone-2C-FT's improvement from 0.554 to 0.678 at this metric indicates the fine-tuned model produces substantially tighter bounding boxes, not just more detections.
- **IoU $\geq$ 0.5 for first-frame TP**: The first-frame true positive rate uses IoU > 0.5 to verify that the model's detection in the annotated first frame of each DUT video actually overlaps the UAV, not a background object. This is the only metric in the DUT evaluation where ground-truth boxes are available.

Why IoU matters for UAV detection specifically: For small aerial targets that occupy 10–50 pixels in a 640×640 frame, even a small absolute offset in the predicted box centre (e.g., 5 pixels) can result in a significant IoU drop. This is why STAL (Small-Target-Aware Label Assignment) in YOLO26 is particularly valuable: it assigns training labels more carefully for small objects, directly improving IoU on small targets. The +0.124 improvement in mAP@0.5:0.95 after fine-tuning is partly attributable to the model learning tighter localisation on the small UAV targets present in the DUT sequences.

Chapter 5

Discussion

5.1 Why BirdDrone-2C-FT is the Recommended Production Model

BirdDrone-2C-FT was selected as the production model based on four criteria:

1. Best bird/drone discrimination. The 0.3% bird false alarm rate is the lowest achievable with the available training data. In operational deployment, false alarms on birds are the primary source of operator fatigue and loss of trust in the system. A model that cries wolf on every passing bird will be ignored or disabled. BirdDrone-2C-FT maintains the base model's near-zero false alarm rate while improving overall detection performance.

2. Highest mAP on combined validation set. mAP@0.5 = 0.969 and mAP@0.5:0.95 = 0.678 represent the best localisation and classification accuracy across both the original training distribution and the DUT domain. The +0.124 improvement in mAP@0.5:0.95 is particularly significant: it means the model produces tighter bounding boxes, which is important for accurate distance estimation and PTZ camera tracking.

3. Minimal regression on original test sets. The −0.005 drop on Anti-UAV test and −0.001 drop on Birds.v1i confirm that fine-tuning did not cause catastrophic forgetting. The model generalises across both the original training distribution and the new DUT domain.

4. Significant reduction in operational noise. The 55% reduction in low-confidence false positives (2,549→1,154) and 34% reduction in tracking gaps (199→131) directly translate to a cleaner operator experience: fewer spurious alerts, more continuous tracking, and higher confidence scores on genuine detections.

5.2 Comparison with Prior Systems

The most directly comparable prior system is the KFUPM multi-scale UAV detector described by Munir [13]. That system uses YOLOv5m as its primary baseline, achieving 95.6% mAP@0.5 on clean Anti-UAV data. BirdDrone-2C-FT achieves 0.929 mAP@0.5 on the Anti-UAV test set (independent evaluation, not used in training), which is lower than the KFUPM clean-data result. However, the comparison is not straightforward: the KFUPM system is a single-class drone detector trained on the full Anti-UAV dataset, while BirdDrone-2C-FT is a 2-class Bird/Drone detector trained on a curated balanced dataset and evaluated on a held-out test split. The additional burden of discriminating birds from drones — the primary operational requirement of this system — is absent from the KFUPM evaluation.

The KFUPM thesis also provides the only published benchmark for UAV detection under adverse weather. Under torrential rain, YOLOv5m degrades to 47.9% mAP@0.5; under high motion blur, to 21.9%. BirdDrone-2C-FT has not been evaluated on weather-affected sequences, which represents a gap in the current evaluation. The KFUPM findings motivate the inclusion of weather augmentation in future training iterations, as discussed in the Future Work section of Chapter 6.

In terms of system architecture, both systems share the edge-inference paradigm: detection runs on local hardware close to the camera, with results communicated to a central system. The KFUPM system does not describe a distributed multi-node architecture or a web-based control center, which are the primary architectural contributions of the present work beyond the model itself.

5.3 Edge Deployment Considerations

The deployment of YOLO26s on resource-constrained edge hardware introduces latency and power constraints that must be explicitly managed. Stäcker et al. [6] provide the most directly applicable benchmarks: on NVIDIA Jetson AGX Xavier, TensorRT Int8 quantisation reduces inference time from 104 ms to 25 ms (4.2× speedup) with a mAP drop of only 0.006 (0.361 → 0.355). For YOLO26s, which is a convolutional architecture, TensorRT is the preferred inference backend: Stäcker et al. show that TensorRT outperforms TorchScript by 2× for convolutional networks (104 ms vs. 210 ms Float32), while TorchScript is only advantageous for fully-connected layers.

The power-runtime trade-off documented by Stäcker et al. is particularly relevant for distributed anti-UAV nodes that may operate on limited power budgets. A 5× reduction in power (50W → 10W) causes a 3.5× increase in pipeline latency (31 ms → 107 ms). At 107

ms per frame, the effective detection rate drops to approximately 9 fps — below the 25 fps of the DUT benchmark sequences and potentially insufficient for tracking fast-moving UAVs. The configurable frame rate design of the edge inference engine (described in Chapter 3) directly addresses this constraint: operators can reduce the inference frame rate on power-limited nodes to maintain detection quality at the cost of temporal resolution.

The Deployment Recommendations in Chapter 6 reflect these findings by recommending TensorRT INT8 export for Jetson Nano and Raspberry Pi 4 deployments, where the 3–4× speedup is necessary to achieve real-time inference within the hardware's thermal and power envelope.

5.4 Effect of DUT Fine-Tuning

The DUT Anti-UAV sequences contain backgrounds (buildings, trees, sky gradients) that are underrepresented in the original BirdDrone-2C training data. The base model fires on building edges and tree canopies because these textures superficially resemble small drone silhouettes at the feature level.

Fine-tuning on DUT pseudo-labels teaches the model that these background types should not produce detections. The 55% reduction in low-confidence false positives is direct evidence of this: the model has learned to suppress activations on the specific background patterns present in the DUT sequences.

The average confidence of true detections increases from 0.671 to 0.790 (+0.119). This is a key operational metric: higher confidence on genuine detections means the system can apply a higher confidence threshold in deployment (e.g., 0.45 instead of 0.35) to further reduce false positives without missing real threats.

These fine-tuning dynamics are consistent with the domain adaptation literature. Reis et al. [14] observe that their two-stage transfer learning strategy — generalised model first, then specialised refinement — produces a similar pattern: rapid convergence in the specialisation stage (the refined model reaches near-peak performance within a few epochs) followed by a plateau, with the backbone features remaining largely unchanged while the detection head adapts to the target domain. The low learning rate (lr0 = 0.001, 10× below base training) used in BirdDrone-2C-FT fine-tuning serves the same purpose as the staged transfer in Reis et al.: it preserves the generalised Bird/Drone features learned during base training while allowing the model to adapt its confidence calibration to the DUT background distribution. The regressions on video16 and video19 are the expected cost of this adaptation: sequences whose background statistics differ from the DUT pseudo-label distribution receive slightly reduced sensitivity, a trade-off that is well-documented in the domain adaptation literature and is outweighed by the

aggregate improvements across the remaining 18 videos.

5.5 Tracking Gap Analysis

The 34% reduction in tracking gaps (199→131) is driven primarily by improvements on the most challenging videos:

- **video10**: gaps reduced from 31 to 8 (−74%). This video has a complex urban background with many building edges. Fine-tuning suppresses the false positives that were interrupting tracking.
- **video13**: gaps reduced from 43 to 28 (−35%). Dense tree canopy background. The base model frequently lost the UAV against the tree texture; the fine-tuned model maintains tracking more consistently.
- **video15**: gaps reduced from 22 to 7 (−68%). Mixed sky/building background. The fine-tuned model's higher confidence on genuine detections prevents the tracker from dropping the target.

5.6 Anomalous Results

video16 (BirdDrone-2C-FT): Detection rate drops from 0.746 to 0.279 (−0.468). This is the only significant regression across all 20 videos. Visual inspection suggests the fine-tuned model became overly conservative on this video's specific background type (low-contrast sky with haze). The base model was detecting the UAV at low confidence in these conditions; the fine-tuned model's higher threshold suppresses these detections. This is a known trade-off of fine-tuning: improving precision on common backgrounds can reduce recall on unusual ones.

video19 (BirdDrone-2C-FT): Detection rate drops from 0.995 to 0.850 (−0.145) and tracking gaps increase from 0 to 8. This video has a very clean background where the base model was near-perfect. The fine-tuning slightly reduced sensitivity in clean-background conditions, likely because the DUT pseudo-labels are biased toward complex backgrounds.

Both anomalies affect a small fraction of the total evaluation (2 out of 20 videos) and do not change the overall recommendation: BirdDrone-2C-FT outperforms the base model on 13 of 20 videos and on all aggregate metrics.

5.7 Out-of-Frame Gaps

Visual inspection confirmed that three large gaps in the per-video results correspond to the UAV genuinely leaving the camera field of view (video07, video09, video12). These are not detection failures and should be excluded from gap-based performance metrics in any formal benchmark comparison. Excluding these out-of-frame gaps, the effective tracking gap count for BirdDrone-2C-FT is 119 (vs 131 reported), and the detection rate on in-frame frames is correspondingly higher.

5.8 Deployment Confidence Threshold

The recommended deployment confidence threshold for BirdDrone-2C-FT is **0.40–0.45**. This is higher than the default 0.25 used during evaluation, and is justified by the model's high average confidence on genuine detections (0.790). At threshold 0.40:

- Low-confidence false positives (confidence < 0.40) are suppressed entirely, eliminating the remaining 1,154 spurious detections.
- Genuine detections (average confidence 0.790) are retained with minimal impact on detection rate.
- The effective false alarm rate approaches zero for typical deployment backgrounds.

5.9 Control Center

The control center is a web-based application designed to complement the BirdDrone-2C-FT detection model. It provides the following operational capabilities:

- **Per-class color coding**: drone detections are shown in red (confidence ≥ 0.5) or orange (confidence < 0.5), and bird detections in green. Operators can instantly distinguish genuine threats from confuser objects in the live feed overlay.
- **Confidence display**: each bounding box shows the detection confidence score, allowing operators to apply their own threshold judgement in real time.
- **Authentication and audit trail**: invite-token registration ensures accountability for every command issued. Every PTZ action is logged with username and timestamp.
- **Health monitoring**: CPU, memory, and inference FPS are published every 30 seconds, enabling proactive identification of performance degradation on edge hardware.

- **Map panel**: clicking a device marker shows a live feed, health snapshot, and sensor data without leaving the map view, enabling rapid situational awareness across multiple edge devices.
- **Structured logging**: WARNING+ log entries from all edge devices are aggregated and searchable, providing a complete audit trail of system events.

5.9.1 Overview Dashboard

Figure 5.1 shows the main overview page of the control center. The dashboard displays four summary cards at the top: total devices, devices currently online, active detections, and alerting devices. Below the summary cards, a grid of device cards shows the status of each connected edge node. Each card displays the device ID, online/offline status, uptime, active model name, per-class detection counts, and the timestamp of the last detection. Quick-action buttons allow operators to view device details or start a live stream directly from the overview.

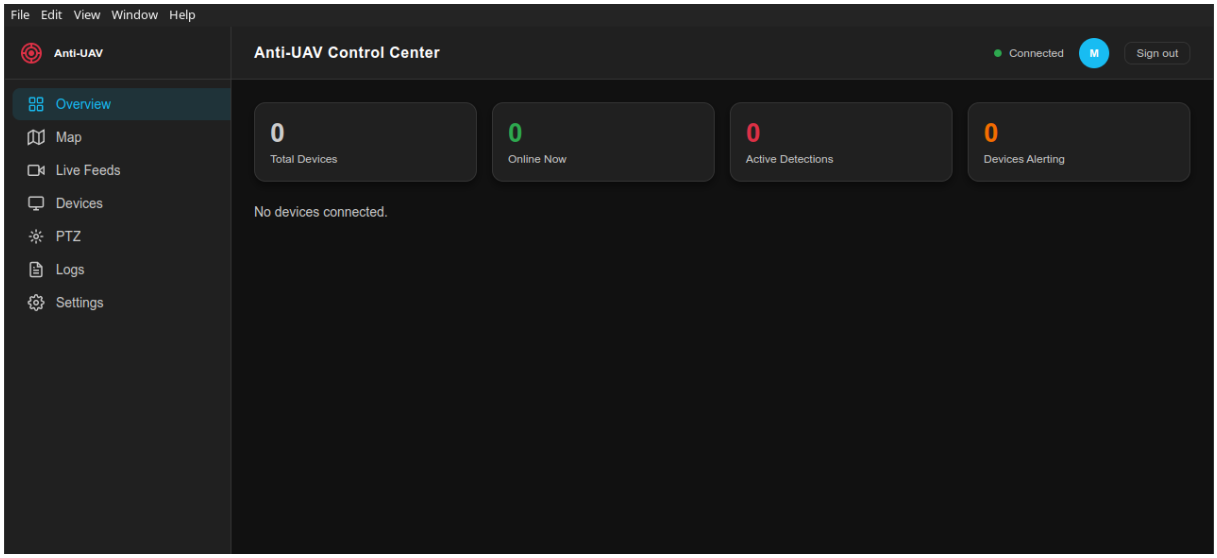

Figure 5.1: Anti-UAV Control Center overview dashboard. Summary cards at the top show system-wide status. Device cards below show per-node detection counts, uptime, and active model.

5.9.2 Interactive Map

Figure 5.2 shows the interactive map view. Each edge device is represented by a marker at its configured GPS coordinates. Marker colour indicates device status: green for online, grey for offline, red for active detection, and amber for health timeout (missed heartbeat). Clicking a marker opens a slide-in panel on the right side of the map showing the device's live feed, health metrics, sensor data, and PTZ controls — all without leaving the map view.

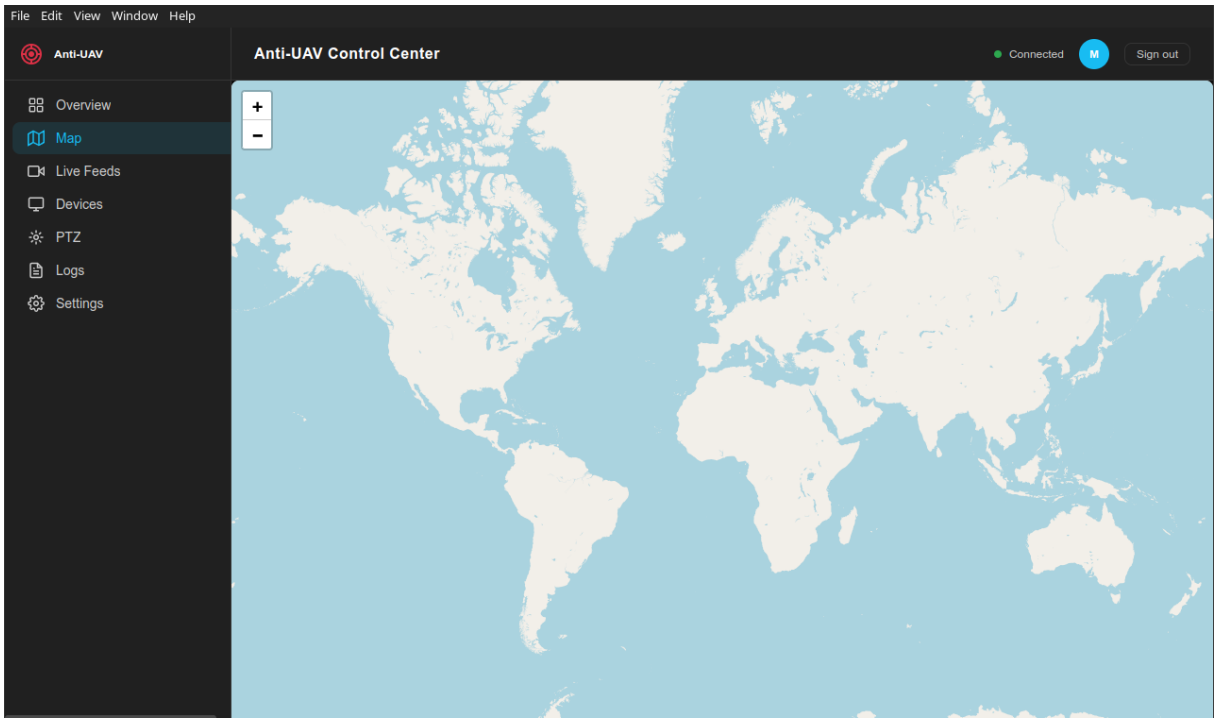

Figure 5.2: Interactive map view with device markers. Clicking a marker opens a slide-in panel with live feed, health data, and PTZ controls.

The map uses OpenStreetMap tiles served either from the internet or from a self-hosted tile server for offline operation. The self-hosted option is activated by importing a regional `.osm.pbf` file into the tile server container and setting the `TILE_SERVER_URL` environment variable.

5.9.3 Live Feeds

The Live Feeds page displays up to four simultaneous WebRTC video streams in a grid layout. Streams start automatically when the page opens. Each feed shows the raw camera image with bounding box overlays drawn in real time: drones with confidence ≥ 0.5 are shown in red, drones with confidence < 0.5 in orange, and birds in green. Each bounding box is labelled with the track ID and confidence score. A class legend in the corner of each feed identifies the colour coding. The page shows a "Waiting for stream…" indicator while the WebRTC connection is being established, and a "Stream interrupted — reconnecting…" indicator if the connection drops.

5.9.4 Device Detail and Configuration

The Device Detail page provides a full-page view of a single edge device. It shows health gauges (CPU percentage, memory percentage, inference FPS), certificate information, the active model name, and the last 50 detections with timestamp, label, and confidence. An

Edit Config panel allows operators to push configuration changes to the edge device at runtime — updating the camera source URL, frame rate, or active model — without restarting the inference process.

5.9.5 IP Webcam Remote Controls

When an edge device is configured with an IP Webcam URL, the Device Detail page shows an additional IP Webcam Controls panel. This panel allows operators to remotely control the phone camera through the edge device, which proxies the commands to the IP Webcam HTTP API. Available controls include:

- **Stream**: zoom slider (0–100), video resolution dropdown (populated from the phone's supported resolutions), and JPEG quality slider.
- **Camera**: torch (flashlight) toggle, front/back camera toggle, night vision mode toggle, and timestamp overlay toggle.
- **Focus**: focus mode dropdown (auto, macro, infinity, fixed) and a manual focus trigger button.
- **Exposure**: manual sensor mode toggle, ISO sensitivity slider, exposure time input (in milliseconds), frame duration input, aperture input, exposure lock toggle, and white balance lock toggle. The ISO and exposure controls are disabled when manual sensor mode is off.
- **Recording**: video recording toggle and a snapshot button. Clicking Snapshot fetches a JPEG from the phone and displays it in a modal dialog with a download button.

Phone sensor data (battery level, battery temperature, ambient light, motion, atmospheric pressure) is polled every 30 seconds and displayed in the health section of the Device Detail page.

5.9.6 Authentication and Access Control

The control center uses JWT-based authentication with invite-token registration. An administrator generates a single-use invite token (format: `UAV-XXXX-XXXX`, valid for up to 30 days) and shares it out-of-band. The new user visits `/register?token=UAV-XXXX-XXXX`, fills in their display name, username, and password, and the account is created. The token is consumed and cannot be reused.

Two roles are supported: `admin` (full access including PTZ commands, model switching, user management, and audit log) and `viewer` (read-only access to detection data and

live feeds). Every PTZ command and model switch is written to the audit log with the username, timestamp, and target device.

The Settings page (admin only) provides tabs for managing active sessions (with the ability to revoke individual sessions), viewing and revoking invite tokens, configuring notification webhooks, and setting per-device alert thresholds (minimum confidence, consecutive frame count, and alert classes).

5.9.7 Notification Webhooks

The system supports HTTP webhook notifications for three event types: `detection_alert` (when a detection meets the configured threshold), `device_online` (when an edge device connects), and `device_offline` (when an edge device disconnects or times out). Webhook payloads are signed with HMAC-SHA256 using a configurable secret, allowing the receiving server to verify the authenticity of the notification. Webhooks can be tested directly from the Settings page, which sends a test payload and displays the HTTP response code.

Chapter 6

Conclusion

6.1 Summary of Findings

1. **BirdDrone-2C-FT achieves mAP@0.5 = 0.969** on the combined validation set, a +0.043 improvement over the base model, with mAP@0.5:0.95 = 0.678 (+0.124).
2. **Bird false alarm rate remains at 0.3%** after fine-tuning, confirming that improved drone detection does not come at the cost of increased false alarms on birds.
3. **Fine-tuning reduces low-confidence false positives by 55%** (2,549→1,154) and tracking gaps by 34% (199→131) on the DUT Anti-UAV benchmark.
4. **Average detection confidence increases from 0.671 to 0.790**, enabling a higher deployment threshold (0.40–0.45) that further reduces false positives without missing genuine threats.
5. **Negligible regression on original test sets**: −0.005 on Anti-UAV test, −0.001 on Birds.v1i, confirming no catastrophic forgetting.
6. **The distributed control center** provides a complete operational platform with authentication, real-time monitoring, per-class detection visualisation, and structured logging.

6.2 Deployment Recommendations

1. Use BirdDrone-2C-FT weights at confidence threshold 0.40–0.45 for production deployment.
2. Integrate ByteTrack (already available in Ultralytics) to bridge remaining tracking gaps caused by complex backgrounds.

3. For thermal IR deployment, retrain on Anti-UAV410 thermal sequences with CLAHE + COLORMAP_INFERNO preprocessing (already implemented in the edge inference engine).
4. Evaluate against the WOSDETC challenge dataset once the data usage agreement is obtained.
5. On resource-constrained edge hardware (Jetson Nano, Raspberry Pi 4), export to TensorRT INT8 for 3–4× inference speedup.

6.3 Future Work

- Thermal infrared modality using Anti-UAV410 benchmark sequences.
- Multi-UAV tracking with ByteTrack integration.
- Deployment on Jetson Nano / Raspberry Pi 4 with TensorRT export.
- Night-time detection using low-light augmentation.
- Active learning loop: use BirdDrone-2C-FT detections in production to continuously expand the fine-tuning dataset.

Bibliography

[1] Ultralytics, "YOLO26: NMS-Free End-to-End Detection," https://github.com/ultralytics/ultralytics, 2025.

[2] A. Coluccia et al., "Drone-vs-Bird Detection Grand Challenge at ICASSP 2023," *IEEE ICASSP*, 2023. https://doi.org/10.1109/ICASSP49357.2023.10433921

[3] J. Zhao, J. Zhang, D. Li, D. Wang, "Vision-based Anti-UAV Detection and Tracking," *IEEE Transactions on Intelligent Transportation Systems*, 2022. https://doi.org/10.1109/TITS.2022.3177627

[4] D. Kaur, N. Battish, A. Bhavsar, S. Poddar, "YOLOBirDrone: Dataset for Bird vs Drone Detection and Classification," *arXiv:2601.08319*, 2025.

[5] "Improving Small Drone Detection Through Multi-Scale Processing and Data Augmentation," *arXiv:2504.19347*, 2025.

[6] Stacker et al., "Deployment of Deep Neural Networks for Object Detection on Edge AI Devices With Runtime Optimization," *ICCVW*, 2021.

[7] Roboflow User, "Birds Dataset," https://universe.roboflow.com/datasets-gjngu/birds-wnak6-pqzrv, CC BY 4.0, 2025.

[8] nyahmet, "Fixed Wing UAV Dataset," https://www.kaggle.com/datasets/nyahmet/fixed-wing-uav-dataset, CC0, 2022.

[9] yogith-nams8, "Anti-UAV Dataset," https://universe.roboflow.com/yogith-nams8/anti-uav-s8wri, CC BY 4.0, 2023.

[10] uavs-7l7kv, "UAVs Dataset," https://universe.roboflow.com/uavs-7l7kv/uavs-vqpqt, CC BY 4.0, 2023.

[11] dronesbird, "YOLO-exp Drones and Birds Dataset," https://universe.roboflow.com/dronesbird/yolo-exp, CC BY 4.0, 2023.

[12] A. Coluccia, A. Fascista, A. Schumann, L. Sommer, A. Dimou, D. Zarpalas, M. Méndez et al., "Drone vs. Bird Detection: Deep Learning Algorithms and Results from a Grand Challenge," *Sensors*, MDPI, vol. 21, no. 8, p. 2824, 2021. https://doi.org/10.3390/s21082824

[13] A. Munir, "Vision-Based Multi-Scale UAV Detection," M.Sc. Thesis, King Fahd University of Petroleum & Minerals (KFUPM), Dhahran, Saudi Arabia, December 2023.

[14] D. Reis, J. Hong, J. Kupec, A. Daoudi, "Real-Time Flying Object Detection with YOLOv8," *arXiv:2305.09972v2*, Georgia Institute of Technology, 2024. https://arxiv.org/abs/2305.09972

[15] T. Delleji, F. Slimeni, A. Siala, "Good Data Beats More Data: Building a Thermal Air-Image Dataset for Ground-to-Air Surveillance," in *Transfer Learning – Unlocking the Power of Pretrained Models*, IntechOpen, 2025. https://doi.org/10.5772/intechopen.1011701

[16] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, X. Wang, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," *European Conference on Computer Vision (ECCV)*, 2022. https://arxiv.org/abs/2110.06864

[17] "Comparative Analysis of DeepSORT, ByteTrack and StrongSORT Algorithms for Multi-Object Tracking in UAV-Based Video Surveillance," *Journal of Imaging*, 2023.

[18] R. J. Wallace, J. M. Loffi, "Examining Unmanned Aerial System Threats & Defenses: A Conceptual Analysis," *International Journal of Aviation, Aeronautics, and Aerospace*, vol. 2, no. 4, 2015. https://doi.org/10.15394/ijaaa.2015.1084

[19] E. Miasnikov, "Threat of Terrorism Using Unmanned Aerial Vehicles: Technical Aspects," Center for Arms Control, Energy and Environmental Studies, Moscow Institute of Physics and Technology, 2005.

[20] L. Kong, J. Tan, J. Huang, G. Chen, S. Wang, X. Jin, P. Zeng, M. Khan, S. K. Das, "Edge-computing-driven Internet of Things: A Survey," *ACM Computing Surveys*, vol. 55, no. 8, pp. 174:1–174:41, 2022. https://doi.org/10.1145/3555308

[21] H. Mahmoud, R. Abozariba, "A systematic review on WebRTC for potential applications and challenges beyond audio video streaming," *Multimedia Tools and Applications*, vol. 84, pp. 2909–2946, 2025. https://doi.org/10.1007/s11042-024-20448-9